# The functional flow of modern Transformer-based architectures: a mathematical cheat-sheet

Akshay Mishra

March 2026

## 1 Mathematical preliminaries and notation

To ensure no prior knowledge is assumed, the following foundational operators and sets are defined:

- $\mathbb{R}^{a \times b}$: A matrix of real numbers with $a$ rows and $b$ columns. This represents a 2D grid of continuous values.
- $\odot$: Element-wise multiplication (Hadamard product). If $C = A \odot B$, then $C_{i,j} = A_{i,j} \times B_{i,j}$. Unlike standard matrix multiplication, this scales corresponding values directly against each other.
- Linear($x$): A standard linear transformation defined as $\text{Linear}(x) = xW + b$, where $W$ is a learnable weight matrix and $b$ is a learnable bias vector. This is the basic building block of neural transformations, projecting data from one dimensional space to another.

—

## 2 The input and embedding layer

Neural networks require continuous numerical representations to process discrete text [12]. You cannot multiply the word "apple" by a weight matrix; it must become an array of numbers.

### 2.1 The embedding matrix

The embedding weight matrix $W_e$ is a learnable dictionary mapping discrete words to continuous vectors. Think of it as a massive lookup table where every row corresponds to a specific word's "meaning" in space.

$$W_e \in \mathbb{R}^{N \times d} \tag{1}$$

**Definitions:**

- $N$: Vocabulary Size (the total number of unique words/tokens the model knows).
- $d$: Embedding Dimension (the size of the continuous vector representing each token). Larger dimensions allow the model to capture more nuanced nuances (e.g., gender, tense, sentiment).

### 2.2 Input transformation

An input sequence of $c$ tokens is represented as a matrix of one-hot vectors, $O$. A one-hot vector contains a 1 at the index of the specific word, and 0s everywhere else.

$$E = O \cdot W_e \in \mathbb{R}^{c \times d} \tag{2}$$

**Definitions and mechanics:**

- $O \in \mathbb{R}^{c \times N}$: The one-hot encoded input matrix.
- $c$: Context Size (sequence length). The number of tokens currently being processed.
- $E \in \mathbb{R}^{c \times d}$: The dense embedding matrix. Mathematically, multiplying the one-hot matrix $O$ by $W_e$ simply "plucks out" the correct rows from $W_e$, assembling them into a sequence of continuous vectors.

### 2.3 Positional encoding (for attention models)

**Concept:** Because standard Transformers process all $c$ tokens in parallel, we must inject sequence information so the model knows the order of words [1]. Without this, "The dog bit the man" and "The man bit the dog" would look mathematically identical. We add a Positional Encoding matrix $P$ to the embeddings:

$$Z_0 = E + P \in \mathbb{R}^{c \times d} \tag{3}$$

Elements in $P$ are calculated using sine (sin) and cosine (cos) functions of varying frequencies:

$$P_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \tag{4}$$

$$P_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{5}$$

**Definitions and mechanics:**

- $pos$: Token position index (0 to $c-1$).
- $i$: Dimension index along the vector (0 to $d/2-1$).
- **Why sine/cosine?** The term $10000^{2i/d}$ creates a geometric progression of wavelengths. Early dimensions in the vector fluctuate wildly (like the seconds hand on a clock), while later dimensions fluctuate very slowly (like the hour hand). This allows the model to easily calculate relative distances between words.
- $Z_0 \in \mathbb{R}^{c \times d}$: The initial sequence matrix that will be passed into the very first architectural block.

—

## 3 Standard Multi-Head Attention (MHA)

This block constitutes the core routing mechanism of the Transformer [1], allowing tokens to share information across the context window $c$.

### 3.1 Defining the input sequence ($Z$)

Let $Z \in \mathbb{R}^{c \times d}$ represent the continuous vector sequence entering the current block.

- For the very first block, $Z$ is exactly the embedded and positionally encoded input: $Z = Z_0$.
- For any subsequent block $l$ (where $l$ goes up to $k$), $Z$ is the final output of the previous block: $Z = Z_{l-1}$.

### 3.2 Linear projections: Queries, Keys, and Values

We split the embedding dimension $d$ into $h$ "heads" to learn different relationships simultaneously. For example, one head might look for grammatical subjects, while another looks for emotional tone.

$$Q_i = ZW_i^Q, \quad K_i = ZW_i^K, \quad V_i = ZW_i^V \in \mathbb{R}^{c \times d_k} \tag{6}$$

**Definitions and the retrieval analogy:**

- $h$: Number of attention heads.
- $d_k = d/h$: Dimension of each head.
- $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$: Trainable weight matrices mapping the input $Z$ to Queries, Keys, and Values.
- **Query ($Q$):** What a token is looking for (e.g., "I am an adjective looking for my noun").
- **Key ($K$):** What a token is (e.g., "I am a noun describing an animal").
- **Value ($V$):** The token's actual underlying meaning/content that will be transferred if a Query and Key match.

### 3.3 Scaled dot-product attention and Softmax

$$\text{head}_i = \text{Softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i \in \mathbb{R}^{c \times d_k} \tag{7}$$

**Definitions and mechanics:**

- $Q_i K_i^T \in \mathbb{R}^{c \times c}$: Raw similarity scores. We take the dot product of every Query against every Key. A high dot product means high relevance. This requires $c \times c$ calculations, creating an $O(c^2)$ computational bottleneck.
- $\sqrt{d_k}$: Scaling factor. If $d_k$ is large, dot products can yield massive numbers. This pushes the Softmax function into regions where gradients approach zero (vanishing gradients). Dividing by $\sqrt{d_k}$ preserves variance and keeps learning stable.
- **Softmax:** Normalizes the $c \times c$ similarity scores into a probability distribution summing to 1.

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{8}$$

- The normalized scores are then multiplied by the Values ($V_i$). This creates a weighted sum: tokens absorb a lot of information from highly relevant tokens, and zero information from irrelevant ones.

### 3.4 Multiple-head concatenation

$$\text{MHA}(Z) = [\text{head}_1 \| \dots \| \text{head}_h] \cdot W^O \in \mathbb{R}^{c \times d} \tag{9}$$

**Definitions:**

- $\|$: The concatenation operator (joining matrices side-by-side). We paste all the individual heads back together.
- $W^O \in \mathbb{R}^{d \times d}$: The trainable output projection matrix that mixes the heads together, synthesizing the different perspectives learned by the parallel heads.

—

## 4 Attention masking (causal and padding)

**Concept:** In practice, the raw similarity scores computed in the Standard Multi-Head Attention block must be filtered before the Softmax function is applied. This prevents tokens from interacting with inappropriate context, such as future words during autoregressive generation (causal masking) or meaningless filler words used to standardize sequence lengths in batches (padding masking).

The standard attention equation from Section 3.3 is modified to include an additive mask matrix $M$:

$$\text{head}_i = \text{Softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}} + M\right) V_i \in \mathbb{R}^{c \times d_k} \tag{10}$$

**Definitions and mechanics:**

- $M \in \mathbb{R}^{c \times c}$: The mask matrix. It mathematically dictates which tokens are allowed to attend to which other tokens in the context window.
- **Causal masking (autoregressive):** During language generation, a token at position $t$ must only look at past tokens and itself, not future tokens. Otherwise, the model would simply "cheat" by looking at the subsequent words during training. The causal mask is a lower-triangular matrix:

$$M_{i,j} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases} \tag{11}$$

  Here, $i$ is the Query token's position (row) and $j$ is the Key token's position (column).
- **Padding masking:** To process multiple sequences of varying lengths in a single training batch, shorter sequences are padded with empty $\langle \text{PAD} \rangle$ tokens to match the maximum length $c$. We must force the model to ignore these dummy tokens. If position $j$ corresponds to a padding token, then $M_{i,j} = -\infty$ for all rows $i$.
- **Why negative infinity ($-\infty$)?** The Softmax function exponentiates its inputs ($e^x$). By adding $-\infty$ to the raw similarity score of a forbidden connection, we evaluate $e^{-\infty} \to 0$. This ensures that the forbidden token receives exactly 0 probability weight, meaning none of its Value ($V$) vector is transferred to the output state.

—

# 5 Sub-quadratic alternatives (linear time $O(c)$)

To avoid the $O(c^2)$ bottleneck of $Q_i K_i^T$, modern architectures compress the sequence history into a fixed-size mathematical state.

## 5.1 Integration into the mathematical flow

In these architectures, the $O(c^2)$ Multi-Head Attention block is completely removed. However, the rest of the network's structure remains identical. The sub-quadratic module simply replaces $\text{MHA}(Z)$, accepting the identical input $Z \in \mathbb{R}^{c \times d}$ and returning an output of the exact same dimensions $\mathbb{R}^{c \times d}$. The concept of formulating attention as an efficient recurrent operation was initially pioneered by linear attention frameworks [14]. To process $Z$ in linear time, these modules read the matrix $Z$ row-by-row sequentially. Let $x_t \in \mathbb{R}^{1 \times d}$ represent the specific row (token vector) of $Z$ at sequence position $t$ (where $t$ goes from 1 to $c$).

## 5.2 State Space Models (SSMs) like Mamba

**Concept:** SSMs track a compressed hidden state $h_t$ over time [2]. Instead of looking back at all previous tokens, the model updates this singular state vector with each new token. For the token vector $x_t$ at step $t$:

$$h_t = h_{t-1}\bar{A} + x_t\bar{B}_t, \quad y_t = h_t C_t \tag{12}$$

**The selective mechanism:** Traditional SSMs use static matrices. Mamba introduces a *selective mechanism* by making the matrices data-dependent (changing based on the current token $x_t$).

$$\Delta_t = \text{Softplus}(\text{Linear}(x_t)), \quad \bar{B}_t = \text{Linear}(x_t), \quad C_t = \text{Linear}(x_t) \tag{13}$$

**Definitions and mechanics:**

- $h_t \in \mathbb{R}^{1 \times n}$: The hidden state row vector. $n$ is the State Expansion Dimension, which determines the capacity of the memory.
- $\bar{A} \in \mathbb{R}^{n \times n}$: The state transition matrix. It controls how the previous hidden state $h_{t-1}$ decays over time.
- $\bar{B}_t \in \mathbb{R}^{d \times n}$: The input projection matrix. Because it is parameterized by $x_t$, the model learns *what* information from the current token is worth memorizing.
- $C_t \in \mathbb{R}^{n \times d}$: The output projection matrix. It reads the hidden state $h_t$ and extracts the relevant features to produce the current output $y_t$.
- $\Delta_t \in \mathbb{R}^d$: The discrete step size. This acts as an information gate. A large $\Delta_t$ forces the model to focus heavily on the new input $x_t$ and update the state. A small $\Delta_t$ allows the model to ignore $x_t$ and preserve the historical context within $h_{t-1}$. *Crucially, $\Delta_t$ determines the continuous-to-discrete transformation*: the continuous baseline matrices $A$ and $B$ are integrated into discrete matrices $\bar{A}$ and $\bar{B}_t$ using rules like $\bar{A} = \exp(\Delta_t A)$ and $\bar{B}_t \approx \Delta_t B_t$. This scales the "force" of the current token update.
- **Softplus($x$):** A smooth, strictly positive activation function defined mathematically as $\log(1 + e^x)$. It ensures the discretization step $\Delta_t$ is always positive, preventing numerical instability during integration.

**Routing to the FFN:** Because the SSM operates token-by-token, the discrete outputs $y_t \in \mathbb{R}^{1\times d}$ produced at each step $t$ are collected and stacked vertically. This forms the full sequence output matrix $Y = [y_1, y_2, \ldots, y_c]^T \in \mathbb{R}^{c\times d}$. This matrix $Y$ explicitly replaces the MHA($Z$) term in the network pipeline. It will be passed forward into the residual and Layer Normalization functions (detailed in Section 5.4) to construct $Z'$, which ultimately serves as the input to the Feed-Forward Network.

### 5.3 Gated Linear Attention (DeltaNet)

**Concept:** Computes $Q(K^TV)$ instead of $(QK^T)V$ to avoid $c \times c$ scaling, using a running memory $S_t$ [3]. By changing the order of matrix multiplication (associative property), the model maintains a fixed-size memory rather than a growing context window. For token $x_t$:

$$g_t = \sigma(\text{Linear}(x_t)), \quad S_t = g_t \odot S_{t-1} + k_t v_t^T, \quad o_t = q_t S_t \tag{14}$$

**Definitions and mechanics:**

- $S_t \in \mathbb{R}^{d_k\times d_v}$: The fixed-size memory matrix. This stores the summarized context of all tokens up to step $t$.
- $k_t \in \mathbb{R}^{d_k\times 1}$ and $v_t \in \mathbb{R}^{d_v\times 1}$: The current token's Key and Value column vectors.
- $k_t v_t^T$: The outer product of the Key and Value. This produces a $d_k \times d_v$ matrix representing the new information to be added to the memory state.
- $\sigma(x)$: The Sigmoid function, defined as $\sigma(x) = \frac{1}{1+e^{-x}}$. It squeezes inputs into a range between $(0, 1)$ to act as a fractional "forget gate" $g_t$, allowing the memory to decay over time. If $g_t$ is close to 0, old memory is wiped. If $g_t$ is close to 1, old memory is fully retained.
- $q_t \in \mathbb{R}^{1\times d_k}$: The current token's Query row vector.
- $o_t = q_t S_t \in \mathbb{R}^{1\times d_v}$: The output for the current step. The Query vector essentially "reads" the running memory state $S_t$ to extract relevant historical information.

**Routing to the FFN:** Just like the SSM, the token-level outputs $o_t$ are evaluated for all $c$ steps and concatenated sequentially to form a full context matrix $O \in \mathbb{R}^{c\times d_v}$. Because the attention heads might have changed the vector width to $d_v$, this matrix is multiplied by a final output projection matrix $W^O \in \mathbb{R}^{d_v\times d}$ to project it back to the expected hidden dimension $d$. This final $c\times d$ matrix is what stands in place of MHA($Z$), continuing forward into Equation 14 to compute the intermediate state $Z'$ for the Feed-Forward Network.

### 5.4 Multi-Head Latent Attention (MLA)

**Concept:** While not strictly linear time in computation, Multi-Head Latent Attention (popularized by DeepSeek) [4] is heavily utilized to circumvent the memory bottleneck of the Key-Value (KV) cache during inference for long sequences. Instead of storing massive $K$ and $V$ matrices for every token across all heads, MLA compresses the token's historical context into a singular, low-dimensional latent vector.

For a token vector $x_t \in \mathbb{R}^{1\times d}$, a shared latent vector is created:

$$c_t^{KV} = x_t W^{DKV} \in \mathbb{R}^{1\times d_c} \tag{15}$$

The content Keys and Values for all $h$ heads are then reconstructed via up-projection:

$$k_t^C = c_t^{KV} W^{UK}, \quad v_t^C = c_t^{KV} W^{UV} \tag{16}$$

**Definitions and Mechanics:**

- $c_t^{KV} \in \mathbb{R}^{1 \times d_c}$: The compressed latent KV vector. $d_c$ is the latent capacity dimension (where $d_c \ll d$). During generation, the model only needs to cache this tiny vector instead of the full Keys and Values, drastically reducing VRAM usage.
- $W^{DKV} \in \mathbb{R}^{d \times d_c}$: The down-projection weight matrix.
- $W^{UK} \in \mathbb{R}^{d_c \times (h \cdot d_k)}, W^{UV} \in \mathbb{R}^{d_c \times (h \cdot d_v)}$: The up-projection matrices that map the latent vector back into the high-dimensional space for all $h$ heads simultaneously.
- **Decoupled RoPE:** Because standard Rotary Positional Embeddings (RoPE) [13] are applied conditionally based on sequence position, they cannot be easily compressed into a static latent vector. MLA explicitly separates positional information by calculating a lightweight positional key $k_t^R = \text{RoPE}(x_t W^{KR})$, which is then concatenated to the recovered content key $k_t^C$ during the final attention calculation.

—

# 6 Network stability operations

## 6.1 Residual connections

**Concept:** Adds the input of a layer directly to its output to prevent gradients from vanishing in deep networks [5]. Think of it as a highway bypass. If a complex sublayer transformation is failing or unnecessary early in training, the original signal can simply bypass it uncorrupted.

$$\text{Output} = x + \text{Sublayer}(x) \tag{17}$$

**Definitions:**

- Sublayer($x$): A generic placeholder representing whichever transformation is currently being applied to $x$ (e.g., MHA, SSM, or FFN).

## 6.2 Dropout regularization

$$\text{Dropout}(x) = x \odot \mathbf{m} \tag{18}$$

**Definitions and mechanics:** [6] $\mathbf{m} \sim \text{Bernoulli}(1-p)$ is a random mask vector containing 1s and 0s. $p$ is the probability of a neuron being set to 0 (dropped). By randomly deactivating features during training, the network is forced to learn redundant, robust representations rather than relying too heavily on any single neuron (preventing "co-adaptation" or overfitting).

### 6.3 Layer normalization (LayerNorm / RMSNorm)

Forces activations to have stable variance so that numbers do not grow infinitely large across many layers [7, 8]. By keeping numerical ranges predictable, the loss landscape remains smooth and gradients do not explode.

$$\text{LayerNorm}(x) = \gamma \odot \left( \frac{x - \mu}{\sqrt{\sigma_{var}^2 + \epsilon}} \right) + \beta, \quad \text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{d}\sum x_i^2 + \epsilon}} \tag{19}$$

**Definitions:**

- $\mu$ is the arithmetic mean (shifts the data to center around zero).
- $\sigma_{var}^2$ is the variance (scales the data to have a spread of 1).
- $\gamma, \beta \in \mathbb{R}^d$ are trainable scale and shift parameters. Once normalized, the network might actually *want* the data to be shifted or scaled for optimal performance; $\gamma$ and $\beta$ allow the network to explicitly relearn a safe shift and scale.
- $\epsilon$ is a tiny constant ($10^{-5}$) to prevent division by zero.

### 6.4 The intermediate state ($Z'$)

Applying these stability functions to the output of our sequence routing block (whether MHA, $Y$ from Mamba, or $O \cdot W^O$ from Gated Linear Attention) gives us the intermediate sequence matrix $Z'$:

$$Z' = \text{LayerNorm}(Z + \text{Dropout}(\text{RoutingModule}(Z))) \in \mathbb{R}^{c \times d} \tag{20}$$

—

## 7 Feed-Forward Network (FFN)

**Concept:** While Attention and SSMs route information *between* different tokens in the sequence, the Feed-Forward Network processes the information *within* each individual token. It operates on each row of $Z'$ independently and identically. Its purpose is to project the token's representation into a higher-dimensional space where complex, non-linear features can be extracted, and then project it back down.

$$\text{FFN}(Z') = \text{ReLU}(Z'W_1 + b_1)W_2 + b_2 \in \mathbb{R}^{c \times d} \tag{21}$$

**Definitions:**

- $W_1 \in \mathbb{R}^{d \times d_{ff}}$: The expansion weight matrix. It projects the vector from dimension $d$ up to $d_{ff}$.
- $W_2 \in \mathbb{R}^{d_{ff} \times d}$: The contraction weight matrix. It projects the vector back down to the standard embedding dimension $d$.
- $b_1 \in \mathbb{R}^{d_{ff}}, b_2 \in \mathbb{R}^d$: Trainable biases.
- $\text{ReLU}(x)$: Rectified Linear Unit [9], an activation function defined as $\max(0, x)$. It converts all negative numbers to zero, injecting the necessary non-linearity into the network. Without non-linearities, stacking multiple linear layers would collapse mathematically into just a single linear layer.

Applying the stability functions to the FFN completes one full architectural block, yielding $Z_{out}$:

$$Z_{out} = \text{LayerNorm}(Z' + \text{Dropout}(\text{FFN}(Z'))) \in \mathbb{R}^{c \times d} \tag{22}$$

This $Z_{out}$ then becomes the input $Z$ for the next layer. This repeats $k$ times.

—

# 8 Mixture of Experts (MoE)

**Concept:** As models scale, computing the dense Feed-Forward Network (FFN) for every token becomes computationally prohibitive. The Mixture of Experts (MoE) architecture [17] replaces the single dense FFN within each block with a collection of independent FFNs called "experts." A routing mechanism dynamically selects only a small subset of these experts to process each token, vastly increasing the model's parameter count (capacity) without a proportional increase in active computation (FLOPs) per token.

## 8.1 The routing mechanism

Let $E$ be the total number of experts, where each expert $i$ is a standard Feed-Forward Network ($\text{FFN}_i$). A gating network computes a probability distribution over the experts for a given input token $x \in \mathbb{R}^{1 \times d}$:

$$G(x) = \text{Softmax}(xW_g) \in \mathbb{R}^{1 \times E} \tag{23}$$

**Definitions:**

- $W_g \in \mathbb{R}^{d \times E}$: The router weight matrix. It projects the $d$-dimensional token vector into an $E$-dimensional space of expert routing scores.
- $G(x)_i$: The routing probability assigned to expert $i$.

The final output of the MoE layer is the probability-weighted sum of the experts' outputs:

$$\text{MoE}(x) = \sum_{i=1}^{E} G(x)_i \cdot \text{FFN}_i(x) \in \mathbb{R}^{1 \times d} \tag{24}$$

## 8.2 Sparse expert computation (Top-$K$)

If the equation above were evaluated naively, the model would still compute all $E$ experts. To achieve true computational sparsity [18], the router only selects the Top-$K$ highest probability experts (typically $K = 1$ or $K = 2$).

$$\text{TopK}(v, K)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } K \text{ elements of } v \\ -\infty & \text{otherwise} \end{cases} \tag{25}$$

The gating probabilities are recalculated using only the retained experts:

$$G_{sparse}(x) = \text{Softmax}(\text{TopK}(xW_g, K)) \tag{26}$$

Because TopK sets the scores of unselected experts to $-\infty$, the Softmax assigns them exactly 0 probability. Thus, their respective $\text{FFN}_i(x)$ transformations are never computed, saving massive amounts of compute.

### 8.3 Load balancing auxiliary loss ($\mathcal{L}_{aux}$)

During training, the routing network often collapses, sending all tokens to a single expert that initializes slightly better than the rest (a self-reinforcing "rich get richer" cycle). To prevent this, MoE models introduce an auxiliary load-balancing loss added to the main cross-entropy loss. For a batch of $T$ tokens, let $f_i$ be the fraction of tokens routed to expert $i$, and $P_i$ be the mean routing probability allocated to expert $i$ across all tokens:

$$f_i = \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}(\text{token } t \text{ routed to expert } i), \quad P_i = \frac{1}{T}\sum_{t=1}^{T} G(x_t)_i \tag{27}$$

$$\mathcal{L}_{aux} = \alpha \cdot E \sum_{i=1}^{E} f_i \cdot P_i \tag{28}$$

**Mechanics:**

- $\mathbf{1}(\dots)$: The indicator function, yielding 1 if the condition is true and 0 otherwise.
- $\alpha$: A scaling hyper-parameter (e.g., 0.01) dictating the strength of the balancing penalty.
- This loss is minimized when the tokens are uniformly distributed ($f_i = 1/E$ and $P_i = 1/E$ for all $i$). If the model heavily favors one expert, $f_i \cdot P_i$ becomes disproportionately large, increasing the penalty.

—

## 9 Final output and loss function

Once the sequence matrix has passed through all $k$ network blocks, it must be mapped back into a language probability format so the model can make its prediction.

### 9.1 Logits and probabilities

$$L = Z_k W_u \in \mathbb{R}^{c\times N}, \quad P = \text{Softmax}(L) \in \mathbb{R}^{c\times N} \tag{29}$$

**Definitions and mechanics:**

- $Z_k \in \mathbb{R}^{c\times d}$: The output of the final $k$-th block. This matrix holds the highly contextualized vector representations of all $c$ tokens in the sequence.
- $W_u \in \mathbb{R}^{d\times N}$: The un-embedding matrix. Also known as the language modeling head, it projects the $d$-dimensional hidden states back into the $N$-dimensional vocabulary space.
- $L \in \mathbb{R}^{c\times N}$: The Logits matrix. These are the raw, unnormalized mathematical scores representing how strongly the model believes each word in the vocabulary is the correct next token.
- $P \in \mathbb{R}^{c\times N}$: The predicted probability matrix. The Softmax function exponentiates the logits and normalizes them, ensuring that the predicted scores for all $N$ words at any given position $c$ sum to exactly 1.0.

### 9.2 Cross-entropy loss ($\mathcal{L}$)

To train the model, we must measure how far off its predicted probabilities $P$ are from the actual correct words.

$$\mathcal{L} = -\sum_{j=1}^{N} Y_j \log(P_j) \tag{30}$$

**Definitions and Mechanics:**

- $Y \in \mathbb{R}^N$: The true target token (one-hot). Because the ground truth is one-hot encoded, $Y_j = 1$ for the correct word's index, and $Y_j = 0$ for all other indices in the vocabulary.
- $\log(P_j)$: The natural logarithm of the predicted probability for the token at index $j$.
- **Mathematical collapse:** Because $Y_j$ is zero everywhere except at the correct token's index, the summation mathematically collapses to just a single term: $\mathcal{L} = -\log(P_{\text{correct}})$.
- **Objective:** By minimizing this negative log-likelihood loss, the optimizer is forced to push the predicted probability of the correct token ($P_{\text{correct}}$) as close to 1.0 as possible.

If the architecture utilizes a Mixture of Experts (MoE) layer, the auxiliary load-balancing loss ($\mathcal{L}_{aux}$) must be added to the standard cross-entropy loss to prevent expert collapse and form the final training objective:

$$\mathcal{L}_{total} = \mathcal{L} + \mathcal{L}_{aux} \tag{31}$$

—

## 10 Optimization (updating the weights)

### 10.1 The trainable parameters ($\theta$)

Let $\theta$ represent the entire set of trainable parameters in the model. This includes:

- $W_e, W_u$: Embeddings.
- $W_i^Q, W_i^K, W_i^V, W^O$: Attention projections (or their SSM equivalents).
- $W^{DKV}, W^{UK}, W^{UV}, W^{KR}$: Multi-Head Latent Attention (MLA) projection matrices.
- $W_1, W_2, b_1, b_2$: FFN weights and biases (including those inside each MoE expert).
- $W_g$: Mixture of Experts (MoE) router weight matrix.
- $\gamma, \beta$: Normalization scale and shift parameters.

### 10.2 Backpropagation (the chain rule)

To update any weight matrix $W \in \theta$, we calculate its gradient. For a standard linear layer where $Y = X \cdot W$:

$$\nabla_W \mathcal{L} = X^T \cdot \frac{\partial \mathcal{L}}{\partial Y} \tag{32}$$

**Definitions:**

- $\nabla_W \mathcal{L}$ is the gradient of the loss with respect to $W$. It points in the direction of steepest ascent (where the error increases the most).
- $\frac{\partial \mathcal{L}}{\partial Y}$ is the error signal passed backward from the layers ahead. Using the chain rule of calculus [15], the network works backward from the final loss, figuring out exactly how much each parameter contributed to the mistake.

### 10.3 Adam optimizer

Adam [10] adjusts the parameters $\theta$ by tracking moving averages of the gradients.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta \mathcal{L}, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta \mathcal{L})^2 \tag{33}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{34}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{35}$$

**Definitions and physical intuition:**

- $\eta$: The Learning Rate (the step size taken in the direction of the gradient).
- $m_t$: First moment (momentum). Tracks the general direction the gradient is pointing. Like a ball rolling down a hill, it builds up speed if it keeps moving in the same direction, helping it power through small bumps (local minima).
- $v_t$: Second moment (velocity/variance). Tracks how violently the gradient is oscillating. If the gradient jumps wildly back and forth across a ravine, $v_t$ gets large. Dividing by $\sqrt{v_t}$ artificially applies "friction," slowing the optimizer down to prevent it from ping-ponging out of control.
- $\hat{m}_t, \hat{v}_t$: Bias-corrected versions of $m_t$ and $v_t$ to prevent initial updates from skewing toward zero.
- $\beta_1, \beta_2$: Exponential decay rates for the moment estimates. Typically 0.9 and 0.999, controlling how quickly old gradients are forgotten.
- $\epsilon$: A tiny constant to prevent division by zero in the denominator.

### 10.4 Muon optimizer & Newton-Schulz iteration

An alternative optimizer specifically designed for internal weight matrices ($W$) [11].

$$W_{t+1} = W_t - \eta \cdot \text{NewtonSchulz}(\nabla_W \mathcal{L}) \tag{36}$$

**Definitions and mechanics:** NewtonSchulz($M$) mathematically transforms an input matrix $M$ into an orthogonal matrix $Q$ (where $Q^T Q = I$, the identity matrix). It is approximated iteratively via:

$$M_{k+1} = \frac{1}{2} M_k (3I - M_k^T M_k) \tag{37}$$

This orthogonalization forces the gradient updates to be mathematically independent across rows and columns. In practice, this prevents the network from learning redundant, overlapping features across different attention heads, making training highly efficient.

—

# 11 Inference workflows and memory constraints

## 11.1 The autoregressive mathematical workflow

**Concept:** During training, the entire sequence of $c$ tokens is processed simultaneously in parallel. However, during inference (text generation), the model operates *autoregressively*, generating one discrete token at a time. The newly generated token is then fed back into the model as the input for the next step.

For a given generation step $t$, the workflow mathematically mirrors the training process but operates on a sequence length of exactly 1 ($x_t \in \mathbb{R}^{1\times d}$). The process from input to decoded token is as follows:

**1. Input embedding:** The discrete token generated from the previous step, $tok_t$, is embedded and positionally encoded to form the input vector:

$$x_t = \text{OneHot}(tok_t) \cdot W_e + P_{(t)} \in \mathbb{R}^{1\times d} \tag{38}$$

**2. Attention calculation & KV-caching:** To process $x_t$ through the attention block without recalculating the entire sequence history, models utilize a **KV-cache** [16]. The new token's Keys and Values ($k_t = x_t W_i^K, v_t = x_t W_i^V$) are appended to the cached matrices of all preceding tokens:

$$K_{1:t} = [K_{1:t-1} \| k_t], \quad V_{1:t} = [V_{1:t-1} \| v_t] \tag{39}$$

The new token's Query ($q_t = x_t W_i^Q$) then attends to this cached history to calculate the attention output:

$$\text{head}_i = \text{Softmax}\left(\frac{q_t K_{1:t}^T}{\sqrt{d_k}}\right) V_{1:t} \in \mathbb{R}^{1\times d_k} \tag{40}$$

The concatenated heads are projected and added to the residual stream to form the intermediate state $x'_t \in \mathbb{R}^{1\times d}$.

**3. Feed-forward network (FFN):** The intermediate state $x'_t$ is processed identically to the training phase (though Dropout is typically disabled during inference):

$$x_{out,t} = \text{LayerNorm}(x'_t + \text{FFN}(x'_t)) \in \mathbb{R}^{1\times d} \tag{41}$$

**4. Final decoding:** The final block's output is projected back into the vocabulary space to produce logits, which are converted to probabilities:

$$L_t = x_{out,t} W_u \in \mathbb{R}^{1\times N}, \quad P_t = \text{Softmax}(L_t) \in \mathbb{R}^{1\times N} \tag{42}$$

A decoding strategy (e.g., greedy argmax or nucleus sampling) selects the highest probability next token, $tok_{t+1}$, from $P_t$, and the cycle repeats.

## 11.2 The KV-cache memory bottleneck

**Concept:** As outlined in the attention calculation above, while KV-caching saves immense computation (FLOPs) by avoiding redundant calculations, it demands massive and ever-growing memory.

**The bottleneck:**

- $q_t \in \mathbb{R}^{1\times d_k}$: The Query vector for just the single newly generated token.
- $K_{1:t}, V_{1:t} \in \mathbb{R}^{t\times d_k}$: The cached Keys and Values up to the current step.
- The physical size of the cache grows linearly ($O(t)$) with the sequence length $t$. For a context window of 100,000 tokens with large batch sizes, the KV-cache easily exceeds the VRAM of multiple top-tier GPUs, completely bottlenecking standard Transformers.

### 11.3 Constant memory in sub-quadratic models

**Concept:** Alternatives like State Space Models (SSMs) and Gated Linear Attention bypass the KV-cache bottleneck entirely. Because they process sequence information sequentially into a fixed-size mathematical state, they achieve an $O(1)$ memory footprint with respect to sequence length during inference.

**Mechanics:**

- **SSMs (e.g., Mamba):** During generation, the model only needs to store the previous hidden state vector $h_{t-1} \in \mathbb{R}^n$. When generating step $t$, it computes $h_t = \bar{A}h_{t-1} + \bar{B}_t x_t$. The historical context is permanently compressed into this $n$-dimensional vector, requiring no growing memory cache.
- **Gated linear attention:** Similarly, the model only caches the fixed-size $d_k \times d_v$ running memory matrix $S_{t-1}$. The new token simply updates this matrix: $S_t = g_t \odot S_{t-1} + k_t v_t^T$.
- **Result:** This constant memory footprint allows sub-quadratic models to generate theoretically infinite sequences on a single GPU without crashing due to Out-Of-Memory (OOM) errors, making them highly attractive for long-context applications.

—

## 12 Typical hyper-parameter values

These are standard configurations balancing memory capacity and computational cost.

| Hyper-parameter | Symbol | Typical Value(s) |
|---|---|---|
| Embedding Dimension | $d$ | 512, 1024, 4096 (Larger = more expressive power) |
| Number of Heads | $h$ | 8, 32, 128 (More heads = finer granular routing) |
| Head Dimension | $d_k, d_v$ | 64 or 128 |
| Context Size | $c$ | 4096, 128k, 1M+ (Standard MHA struggles past 8k context) |
| Number of Blocks | $k$ | 12, 24, 96 |
| FFN Hidden Dimension | $d_{ff}$ | $4 \times d$ (Standard expansion ratio) |
| SSM State Dimension | $n$ | 16, 64, 128 |
| MLA Latent Dimension | $d_c$ | 512 |
| Number of Experts (MoE) | $E$ | 8, 16, 64, 256 |
| Active Experts (Top-$K$) | $K$ | 1, 2, 4, 8 |
| MoE Aux Loss Coefficient | $\alpha$ | 0.01 or 0.02 |
| Dropout Rate | $p$ | 0.0 to 0.1 (10%) |
| Adam Learning Rate | $\eta$ | $1 \times 10^{-4}$ to $6 \times 10^{-4}$ |
| Adam $\beta_1, \beta_2$ | - | 0.9, 0.95 or 0.999 |
| Vocabulary Size | $N$ | 32,000 to 128,000 |

—

## References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

[2] Gu, A. and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.

[3] Yang, S., Kautz, J. and Hatamizadeh, A. (2024). Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*.

[4] DeepSeek-AI (2024). DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*.

[5] He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).

[6] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1), pp.1929-1958.

[7] Ba, J.L., Kiros, J.R. and Hinton, G.E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.

[8] Zhang, B. and Sennrich, R. (2019). Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32.

[9] Nair, V. and Hinton, G.E. (2010). Rectified linear units improve restricted boltzmann machines. *Proceedings of the 27th international conference on machine learning (ICML-10)* (pp. 807-814).

[10] Kingma, D.P. and Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

[11] Jordan, K., Bernstein, J., Newhouse, L., Boza, V., Jin, Y. et al. (2024). Muon: An optimizer for hidden layers in neural networks. *Published online*.

[12] Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

[13] Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B. and Liu, Y. (2021). RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.

[14] Katharopoulos, A., Vyas, A., Pappas, N. and Fleuret, F. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. *International conference on machine learning* (pp. 5156-5165).

[15] Rumelhart, D.E., Hinton, G.E. and Williams, R.J. (1986). Learning representations by back-propagating errors. *nature*, 323(6088), pp.533-536.

[16] Shazeer, N. (2019). Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*.

[17] Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G. and Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.

[18] Fedus, W., Zoph, B. and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1), pp.5232-5270.